claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_b0e273c7-38a\agent-a137c55b9b4939ec0.jsonl`
- SHA-256: `cfc8e9183a0a0ce528eb5881eae806166049fe7778caf27504554e1947c42e5d`
- Source modified: `2026-06-21T08:00:43+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_b0e273c7-38a`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`

Transcript

1. user (2026-06-21T07:56:51.763Z)

```
You are reviewing milestone M2 of COMPUTE_DECOUPLED_SERVING in the repo at
C:/Users/avidu/Projects/badminton-highlight-indexer.
Spec: docs/COMPUTE_DECOUPLED_SERVING/README.md §7 row M2 = "input_backend/input_root; wire main
CLI/API input through StorageRef.parse_uri (local=identity); storage-fake tests". M2 must be
SAFE and
default-OFF: with the default config (storage.input_backend='local', input_root=''), the loaded
input
path must be byte-identical to before ? a bare/local path is read in place with NO copy. A
bucket/Drive
URI (gs://, s3://, gdrive://) is fetched to a local scratch file before processing.

The change (read all of these + git diff against origin/master):
- backend/config/models.py: StorageConfig gains input_backend + input_root.
- backend/storage/ref.py: resolve_input_uri() (precedence: explicit scheme > anchored local path
>
  input_root join > input_backend prefix > bare-local).
- backend/storage/__init__.py: acquire_local_input() (local=identity, remote=mkdtemp+fetch) +
  input_is_local().
- backend/main.py: _execute_processing() (API/job path) + the CLI --video-path block both
  acquire-then-use the local path (keeping the original URI for the DB record + report);
  /api/process skips the local os.path.exists() gate for a non-local URI.
- tests: tests/test_input_source.py (new) + 3 tests in tests/test_api_endpoints.py.

Reference the worker's already-shipped pattern (backend/worker/__main__.py:91-105 cmd_infer) and
the
path-safety helpers (backend/utils/paths.py: validate_input_path / allowed_video_root). Ground
EVERY
claim in code with file:line. Give the exact triggering input + wrong-vs-right behaviour. Prefer
FEWER,
HIGH-CONFIDENCE findings. A clean bill is a valid, good result.

LENS: resolve_input_uri / acquire_local_input correctness. Enumerate
edge cases: empty string ""; a name with no extension (ref.name fallback to "in.mp4"); a key
ending in
"/"; input_root with vs without trailing slash; an explicit scheme that ALSO matches
input_backend; a
gdrive:// path; "gs://bucket" with no key. Does the join produce a valid URI parse_uri round-
trips? Is
input_is_local consistent with acquire_local_input (same resolution ? same backend)? Does the
worker's
existing storage_cfg shape (config.storage.model_dump()) match what acquire_local_input forwards
to
fetch? Any divergence between the two code paths (worker vs main) that could yield different
```

bytes?

2. assistant (2026-06-21T07:56:54.092Z)

I'll review M2 of COMPUTE_DECOUPLED_SERVING. Let me start by reading the spec and the relevant code.

3. user (2026-06-21T07:56:57.824Z)

9c64ac5 docs(serving): mark §7 tracker M1 ? + changelog (PR #279)
368155f feat(serving): M1 ? deployment.profile + compute_target config (pure, additive, default-OFF)
f518e97 docs(serving): M0 APPROVED + lock D13-D15 (switchable profiles, Drive Northstar, consent-not-surprise) (#278)
---BRANCH---
feat/serving-m1-deployment-profile

4. user (2026-06-21T07:56:58.309Z)

1 # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3 > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** · **Owner:** Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
4 > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/past_projects/` when DONE)
5 > **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory `current-state` + `docs/NEXT_STEPS.md` once work starts.
6 > **Relation to existing docs:** the productionization successor to the archived `DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from [PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7 > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed **compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an afterthought.
8 > **Honesty note:** claims tagged `[verified]` (checked against code by the design workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10 ---
11
12 > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user *confirms* (D15).
13
14 ## 0. TL;DR
15 The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17 ## 1. Problem & goal
18 - **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local mode, with the **core (detect+stitch) unaffected** by which one runs.
19 - **The two user-facing outcomes** (owner, 2026-06-21):

20   - **(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from **gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching** (stitching is compute-intensive ? run it where its cost is accounted for).
21   - **(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local GPU**, run inference **and** stitching **completely locally**.
22   - **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup choice**, never a code branch in the core.
23
24   ## 2. Decisions locked
25
26 | # | Decision | Source | Implication |
27 |---|----------|--------|-------------|
28 | D1 | The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch inside it**. | design `[verified]` | All profile difference lives in config + the two registries. |
29 | D2 | **Two registries:** `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`. | ADR-014 | Realizes PLATFORM_ARCHITECTURE's `local\|hosted\|byo_worker`. |
30 | D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31 | D4 | In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real cost. |
32 | D5 | **Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory); **detect** is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the rally/window level, never CSV bytes. |
33 | D6 | **Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-convention]] | Zero-regression discipline. |
34 | D7 | **Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that ~$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |
35 | D8 | **Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |
36 | D9 | **Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run **ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |
37 | D10 | **Free-tier "data-for-reels" trade:** free reels in exchange for **training rights** (uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data + **auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads **gated to M9** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel. |
38 | D11 | **Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39 | D12 | **Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) + `data_pipeline/` + the eval/experiment harness. |
40 | D13 | **Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is *between* jobs. |
41 | D14 | **NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a

Google Drive video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. | owner 2026-06-21 | Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |
42 | D15 | **Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |
43
44 ## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*
45 This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:
46
47 | ADR / doc | Contributed | Subtlety carried into this project |
48 |---|---|---|
49 | **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |
50 | **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**. |
51 | **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class. |
52 | **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |
53 | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture | **This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**. |
54 | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |
55 | **ADR-013** | Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only | The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target. |
56 | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not** supersede them. |
57 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |
58 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |
59 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
60 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel parity finding** | Why D5 (parity at window level, not CSV bytes). |
61
62 ## 4. Design / architecture
63 See [`architecture.svg`](architecture.svg) for the picture. **The core never changes; the box around it does.**
64
65 ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
66 1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ? CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
67 2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?) ? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same trajectory + config = byte-identical candidate windows. Used by every runner *and* the eval/regression stack verbatim.
68 3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
69

70   **Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).
71
72   ### 4.2 The deployment profiles
73 | Profile | Compute | Storage | Orchestration | When |
74 |---|---|---|---|---|
75 | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main` or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |
76 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |
77 | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free dev |
78 | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |
79
80   ### 4.3 Profile selection (the "startup/setup config") `[design]`
81   ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-detect in `load_config`:
82   - `deployment.profile ? {truly-local | cloud-serving | cpu-mock}` (empty = today's manual knobs, **fully backward-compatible**).
83   - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy), **STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
84   - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env (`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob stays individually overridable.
85   - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else `torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).
86   - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads `config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
87
88   ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
89   DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) + `wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim_due_job` already supports both.)
90
91   ### 4.5 Reuse map ? most of the seams already exist `[verified]`
92   **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam); `StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.

93
94   ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
95   Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:
96   1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-consent auto-delete path.
97   2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).
98   3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
99   4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
100   This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.
101
102   ## 6. Honest limits ? what this does NOT do / prove
103   - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity** for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).
104   - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.
105   - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with **Gemini handoff ON** until the owned model clears a floor (open Q).
106   - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.
107
108   ## 7. Deliverables & progress tracker ? **source of truth**
109   Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off `origin/master`, no stacking. Default-OFF where it touches core callers.
110
111 | ID | Deliverable | Depends | Gated? | Status | PR |
112 |----|-------------|---------|--------|--------|----|
113 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? | **? APPROVED 2026-06-21** |
114 | M1 | `deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? | [#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
115 | M2 | `input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? | ? |
116 | M3 | Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? | ? |
117 | M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 | safe (cuda default unchanged) | ? | ? |
118 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | ? |
119 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | ? |
120 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |
121 | M8 | Per-job **cost ledger** (v_ migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | ? |
122 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |
123 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |

| **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
| **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |

Testing strategy is a first-class deliverable ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).

8. Open questions ? owner *(blocking-gates vs nice-to-knows)*

? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:

1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).
5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).
6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).

Still open (non-gating, smaller):

2. **Stitch placement:** always co-located in the detect job (simplest, one metered unit) vs split to a cheap CPU Cloud Run service when bursty? *Default co-located unless metrics say otherwise.*
7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264 (determinism/parity)?
9. **Input cap:** is `<2GB` a hard client reject, or accept to the 4GB upload cap and route larger files elsewhere?
10. **Directory name:** `COMPUTE_DECOUPLED_SERVING/` (chosen) vs keep `CLOUD_RUN_GPU_SERVING/` ? confirm.

9. Definition of done

- [ ] M1?M4 land (config + input + output-base + device layer), each green on golden cross-OS + experiment no-regression.
- [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.
- [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog / DEPLOY_REPORT** with a real sample run.
- [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.
- [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented adult jobs; the owned model is on a measured path to the D11 floor.
- [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.

10. References

ADR lineage: ADR-005/008/009/010/011/012/013 ? **ADR-014** (`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`, `HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`, `LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:** `docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`, `stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`, `backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`, `backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-mappers ? synthesis; 58 findings).

```
154
155     ---
156     ### Changelog
157     - 2026-06-21 ? created from the design workflow output; M0 in progress.
158     - 2026-06-21 ? **M0 APPROVED**; **M1 landed**
([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach).
Suite 1068 green, cov 84.11%.
159
```

5. assistant (2026-06-21T07:57:03.223Z)

This is M2, but the current branch is the M1 branch. Let me look at the actual M2 changes. The task says to review the diff against origin/master. Let me find where the M2 work lives.

6. user (2026-06-21T07:57:05.506Z)

```
 M backend/config/models.py
 M backend/main.py
 M backend/storage/__init__.py
 M backend/storage/ref.py
 M tests/test_api_endpoints.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_input_source.py
===STASH===
===BRANCHES===
+ bhi-execpolicy
  chore/consolidate-fusion-schema-predicate
  docs/adr-012-gpu-cloudrun
  docs/archive-decoupled-compute
  docs/archive-decoupled-v2
  docs/compute-decoupled-serving
  docs/court-floor-rally-end
  docs/decouple-scope-defer
  docs/deploy-report-polish
  docs/desktop-app-plan
  docs/doc-status-register
  docs/drop-digitalocean-gcp-only
  docs/findings-1-2-pervideo-nextsteps
  docs/gcp-deploy-report
  docs/gcp-pivot-runbook
  docs/golden-regression-fixtures-plan
  docs/lock-m0-decisions
+ docs/lovo-resweep-n13
  docs/m0-approved-d13-d15
  docs/m2-finalized-report
  docs/point-i18n-ui-to-khelsutra
+ docs/rally-gap-closure
+ docs/recording-single-match
  docs/restructure-field-roora-data
+ dryrun/cuj-local
  feat/api-m1-health-downloadurl
  feat/gdrive-storage-backend
  feat/highlights-runner
  feat/local-no-ai-segmenter
  feat/long-rally-lens
  feat/native-wasb-runner
  feat/prominent-court-gate
* feat/serving-m1-deployment-profile
+ feat/w1-hard-cap-fallback
+ feat/w2-never-empty-safeguard
  fix/compiler-dedup-overlapping-segments
```

```
  fix/fusion-loaders-mixed-schema
  fix/gcp-teardown-default-delete
  fix/gemini-loud-chunk-failures
  fix/gemini-per-video-chunk-dir
===DIFF STAT vs origin/master===
 backend/config/__init__.py               |   5 +
 backend/config/loader.py                 | 100 ++++++++-
 backend/config/models.py                 |  21 ++
 backend/config/presets.py                | 120 ++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md |   3 +-
 tests/test_config_deployment.py          | 371 +++++++++++++++++++++++++++++++
 6 files changed, 617 insertions(+), 3 deletions(-)
```

7. assistant (2026-06-21T07:57:09.172Z)

The M2 work is uncommitted in the working tree (modified files + new `tests/test_input_source.py`). The current branch HEAD has the M1 changes committed. The M2 changes are the uncommitted modifications. Let me read all the relevant files for the M2 review.

8. user (2026-06-21T07:57:09.877Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
```

```
        bucket: str
        key: str
        uri: str

        @property
        def name(self) -> str:
            return self.key.rstrip("/").rsplit("/", 1)[-1]


    def parse_uri(uri: str) -> StorageRef:
        """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.

        A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
        have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
        """
        s = str(uri)
        m = _SCHEME_RE.match(s)
        if not m:
            return StorageRef("local", "", s, f"local://{s}")
        scheme = m.group(1).lower()
        rest = s[m.end():]
        if scheme == "gs":
            scheme = "gcs"
        if scheme in ("local", "gdrive"):
            # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
            return StorageRef(scheme, "", rest, s)
        if scheme in _KNOWN:
            bucket, _, key = rest.partition("/")
            return StorageRef(scheme, bucket, key, s)
        raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")


    def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
        """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).

        Precedence (first match wins):
          1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
          2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
          3. A bare/relative name + a non-empty ``input_root`` ? joined under it
             (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
          4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
             ``"gcs://bucket/x"``).
          5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).

        So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
        path used today, preserving byte-for-byte behaviour.
        """
        s = str(raw)
        if _SCHEME_RE.match(s):           # 1. explicit scheme wins
            return s
        # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
        #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
        #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
        if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
```

```
93              return s
94          if input_root:                    # 3. join a relative name under the configured
root
95              return input_root.rstrip("/") + "/" + s.lstrip("/")
96          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
97              return f"{input_backend}://{s}"
98          return s                          # 5. bare/relative local path ? unchanged
99
```

9. user (2026-06-21T07:57:13.294Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13      from __future__ import annotations
14
15      import os
16      import tempfile
17      from typing import Any, Dict, Iterator, Optional
18
19      from backend.storage.base import StorageBackend
20      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22      __all__ = [
23          "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24          "get_storage", "fetch", "put", "list_uris",
25          "acquire_local_input", "input_is_local",
26      ]
27
28
29      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31          if storage_config is None:
32              return {}
33          if hasattr(storage_config, "model_dump"):
34              return storage_config.model_dump()  # type: ignore[no-any-return]
35          return dict(storage_config)
36
37
38      def input_is_local(raw: str, storage_config: Any = None) -> bool:
39          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
40          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
41          that must be fetched first. Lets callers keep their local-only checks
(os.path.exists,
42          allowed_video_root) for the local case and skip them for a remote source."""
43          sc = _storage_dict(storage_config)
44          uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
45                                  input_root=sc.get("input_root", ""))
46          return parse_uri(uri).backend == "local"
```

```

    def acquire_local_input(raw: str, storage_config: Any = None, *,
                            scratch_parent: "str | None" = None) -> str:
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The scratch copy is left in place (a pure intermediate, regenerable
from the
        source) ? same as the worker's ``cmd_infer`` pull, so cache-hygiene tooling reclaims
it."""
        sc = _storage_dict(storage_config)
        uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
                                input_root=sc.get("input_root", ""))
        ref = parse_uri(uri)
        if ref.backend == "local":
            return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        return fetch(uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage
            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage
            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage
            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage
            return GDriveStorage
        raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")


    def get_storage(backend: Optional[str] = None, *,
                    config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
```

```
100         (no copy); for s3/gcs it downloads to ``dst``."""
101         ref = parse_uri(uri)
102         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
103
104
105     def put(local_path: "str | os.PathLike[str]", uri: str, *,
106             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
107         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
108         ref = parse_uri(uri)
109         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
110         return uri
111
112
113     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
114         """Yield the URI of every object under ``prefix_uri``."""
115         ref = parse_uri(prefix_uri)
116         for r in get_storage(ref.backend, config=config).list(ref):
117             yield r.uri
118
```

10. assistant (2026-06-21T07:57:16.188Z)

Now let me read the config models, main.py changes, and the worker pattern.

11. user (2026-06-21T07:57:16.979Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
```

```
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
73          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
74          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
75          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
76          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
77          keep_frames: bool = False
78          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79          min_free_gb: float = 0.0
80          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
81          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
82          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
83          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
84          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
85          # Tri-state:
86          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
87          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
```

```
88          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
89          stream_video: Optional[bool] = None
90
91
92      class FusionConfig(BaseModel):
93          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
94
95          Every default reproduces control behaviour: the fusion segmenter persists the
96          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
97          true ? the person-cue features, but it does NOT gate the served handoff unless a
98          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
99          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
100         supplied ? off-court persons (spectators / adjacent courts) are excluded.
101         """
102         # Which trajectory substrate seeds the windows (shuttle stays primary).
103         substrate: Literal["wasb", "tracknet"] = "wasb"
104         # Windowing velocity threshold for the fusion family (the engine reads
105         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107         velocity_threshold: float = 0.02
108         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109         # enabled ? so the default path never imports torch / touches the GPU.
110         cue_flags: dict[str, bool] = Field(default_factory=dict)
111         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
113         min_crossings: int = Field(default=0, ge=0)
114         # Served Gate B: when the person cue is on, drop windows with median in-court
players
115         # < this. 0 = OFF.
116         min_players: int = Field(default=0, ge=0)
117         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
118         court_polygon: Optional[list[list[float]]] = None
119         # Net line for the person two-sided-motion split (person features only ? the shuttle
120         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
121         net_axis: Literal["x", "y"] = "y"
122         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
123
124
125     class IndexingConfig(BaseModel):
126         default_segmenter: str = "yolo_hybrid"
127         use_gpu: bool = True
128         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
129         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
130         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
131         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
132         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
133         detector_impl: str = "auto"
134         motion_threshold: float = 0.08
135         min_segment_duration: float = 3.0
136         dead_time_merge_gap: float = 2.0
137         chunk_duration: float = 90.0
138         chunk_overlap: float = 10.0
139         detector_model: str = "fasterrcnn_resnet50_fpn"
140         detector_score_threshold: float = 0.5
141         yolo_velocity_threshold: float = 0.15
142         yolo_frame_skip: int = 3
143         yolo_tracker: str = "bytetrack.yaml"
```

```
144         yolo_prefetch_workers: int = 2
145         min_action_window_duration: float = 2.0
146         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
147         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
148         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
149         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
150         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
151         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
152         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
153         max_window_duration: float = 6.0
154         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
155         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
156         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
157         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
158         # Only cuts (recall can't drop). OFF by default until measured/promoted.
159         window_hard_cap: bool = False
160         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
161         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
162         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
163         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
164         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
165         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
166         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
167         window_inactivity_end: float = 1.5
168         inactivity_rally_thresh: float = 0.20
169         inactivity_min_density: float = 0.30
170         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
171         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
172         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
173         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
174         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
175         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
176         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
177         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
178         window_blob_fallback: bool = False
179         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
180         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
181         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
```

```
active
182         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
183         window_blob_factor: float = 4.0
184         log_yolo_candidates: bool = True
185         store_yolo_candidates: bool = True
186         ai_handoff_padding: float = 2.0
187         ai_max_workers: int = 5
188         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
189         # chunks of a high-bitrate (4K) source blow past the provider upload cap
190         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
191         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
192         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
193         # Matches gemini_refine's --clip-scale-width default.
194         ai_chunk_scale_width: int = 1280
195         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
196         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
197         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
198         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
199         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
200         skip_ai_handoff: bool = False
201         # Optional debug knob: trim every video to the first N seconds before processing.
202         debug_duration: Optional[float] = None
203
204         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
205         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
206         fusion: FusionConfig = Field(default_factory=FusionConfig)
207
208         @field_validator("default_segmenter")
209         @classmethod
210         def segmenter_must_be_registered(cls, v: str) -> str:
211             # Registry check happens at runtime in main/segmenter selection.
212             # Here we just allow any string for forward compatibility.
213             return v
214
215         @model_validator(mode="after")
216         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
217             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
218             # step makes `while t < duration: t += step` loop forever, growing memory before
any
219             # GPU work. Reject the bad config at load time instead.
220             if self.chunk_overlap >= self.chunk_duration:
221                 raise ValueError(
222                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
223                     f"({self.chunk_duration}); otherwise chunking never advances."
224                 )
225             return self
226
227
228     class OutputConfig(BaseModel):
229         default_highlights_name: str = "highlights.mp4"
230         db_name: str = "sports_indexer.db"
231         # Directory where the DB, highlight reels, and processed clips are written.
232         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
233         output_dir: str = "output"
234
235
236     class WatermarkConfig(BaseModel):
237         """Branding overlay burned into each highlight clip during the per-clip re-encode
```

```
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
        # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
        # local detector over-fires and its false positives are mostly SHORT (motion blips,
        # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
        # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
        min_rally_duration: float = Field(default=0.0, ge=0.0)
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]


```

```
295     class StorageConfig(BaseModel):
296         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
302         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304         output_root: str = ""
305         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
306         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
307         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
308         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
309         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
310         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
311         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
312         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
313         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
314         input_root: str = ""
315         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
316         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
317         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
318         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
319         # the legacy flat layout is used unchanged.
320         single_folder_per_video: bool = False
321         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
322         workspace_root: str = ""
323         # Cloud namespace prefix under the root so per-video folders sit tidily beside
324         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
325         workspace_prefix: str = "workspaces"
326         local: dict[str, Any] = Field(default_factory=dict)
327         s3: dict[str, Any] = Field(default_factory=dict)
328         gcs: dict[str, Any] = Field(default_factory=dict)
329         gdrive: dict[str, Any] = Field(default_factory=dict)
330
331
332     class DeploymentConfig(BaseModel):
333         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
334         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
335
336         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
337         behaves exactly as before this section existed. A profile is opt-in (``config.json``
338         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
339         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
```

```
340         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
341         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
342
343         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
344
345         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
346         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
347         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
348         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
349         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
350
351
352     class AppConfig(BaseModel):
353         """Root configuration object.
354
355         All runtime configuration should be accessed via an instance of this class
356         (or the loader) rather than raw dict lookups.
357         """
358
359         sport: Sport = "badminton"
360         ai_provider: str = "gemini"
361         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
362         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
363         ai_model: str = "gemini-flash-latest"
364
365         validation: ValidationConfig = Field(default_factory=ValidationConfig)
366         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
367         output: OutputConfig = Field(default_factory=OutputConfig)
368         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
369         security: SecurityConfig = Field(default_factory=SecurityConfig)
370         logging: LoggingConfig = Field(default_factory=LoggingConfig)
371         storage: StorageConfig = Field(default_factory=StorageConfig)
372         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
373
374         # Allow extra keys during transition so existing config.json files do not break.
375         model_config = {
376             "extra": "allow",
377             "validate_assignment": True,
378         }
379
380         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
381         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
382         # attribute access / the typed config. A nested *section* is returned as a plain
dict
383         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
384         # section model instead breaks it (a model has no .get), which bit every validator
on
385         # real runs. Scalar leaves are returned as-is.
386         def get(self, key: str, default: Any = None) -> Any:
387             _MISSING = object()
388             val: Any = _MISSING
389             if key in type(self).model_fields or hasattr(self, key):
390                 val = getattr(self, key, _MISSING)
391             if val is _MISSING:
392                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
393                 # discovered dynamically from the model fields, so a future top-level
section
394                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
```

```
hardcoded list.
395                 for fname in type(self).model_fields:
396                     section = getattr(self, fname, None)
397                     if isinstance(section, BaseModel) and hasattr(section, key):
398                         val = getattr(section, key)
399                         break
400             if val is _MISSING:
401                 return default
402             if isinstance(val, BaseModel):
403                 return val.model_dump(mode="json")
404             return val
405 
406         def __getitem__(self, key: str) -> Any:
407             val = self.get(key)
408             if val is None and key not in type(self).model_fields:
409                 # Re-raise like a real dict for unknown top-level
410                 raise KeyError(key)
411             return val
412 
413 
414     # Convenience type alias for callers that want to annotate
415     Config = AppConfig
416 
```

12. assistant (2026-06-21T07:57:19.925Z)

Now let me read the main.py changes and the worker pattern. Let me get the diff for main.py and look at the worker.

13. user (2026-06-21T07:57:20.992Z)

```
diff --git a/backend/main.py b/backend/main.py
index 942217e..33c7995 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -32,6 +32,7 @@ from backend.utils.paths import resolve_under_root, safe_output_filename,
valida
 from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  # new
typed config
 from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
+from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)

 app = FastAPI(title="Sports Highlight Indexer API")

@@ -151,13 +152,18 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
     notify = progress or (lambda stage: None)

     notify("hashing")
-    video_id = compute_video_id(req.video_path)
+    # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity
?
+    # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
+    # req.video_path (the source URI) is kept for the DB record + report; only the file-reading
+    # consumers (hash / validators / segmenter) use the resolved local path.
+    local_video = storage.acquire_local_input(req.video_path, getattr(global_config, "storage",
None))
+    video_id = compute_video_id(local_video)
     was_reingest = db_instance.get_video(video_id) is not None

     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the report)
     notify("validating")
     logger.info(f"Running validations for {req.video_path}...")
-    val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
```

```
+    val_results, val_context = registry.run_all_with_context(local_video, global_config)
     failures = [r for r in val_results if not r.passed]

     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
@@ -205,7 +211,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
         segmenter = segmenter_cls(db_instance, global_config)
         try:
-            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
+            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
         except Exception:
             # A raising segmenter must not leave half-written rows: restore the pre-run
             # state (same destroy-after-confirm contract as the Failure branch), then let
@@ -265,8 +271,13 @@ def process_video(req: ProcessRequest):
         validate_input_path(req.video_path, allowed_root=allowed_root)
     except ValueError as e:
         raise HTTPException(status_code=400, detail=str(e)) from e
-    if not os.path.exists(req.video_path):
-        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
+    # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive URI can't
be
+    # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it (a
fetch miss
+    # fails the job loudly). validate_input_path above still runs for every input: its null-
byte guard
+    # always applies, and a configured allowed_video_root (hosted mode) rejects a non-local
URI.
+    if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
+        if not os.path.exists(req.video_path):
+            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
     if req.segmenter and not segmenter_registry.get(req.segmenter):
         available = list(segmenter_registry.list_available().keys())
         raise HTTPException(
@@ -577,16 +588,20 @@ def main():
     # CLI processing mode (no web UI needed)
     if args.video_path and not args.server:
         print(f"[CLI] Processing video file: {args.video_path}")
-        if not os.path.exists(args.video_path):
+        # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
+        # The local-existence fast-fail only applies to a local input; a remote fetch fails
loudly.
+        storage_cfg = getattr(global_config, "storage", None)
+        if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
             print(f"[FAIL] Video file not found: {args.video_path}")
             return
+        local_video = storage.acquire_local_input(args.video_path, storage_cfg)

-        video_id = compute_video_id(args.video_path)
+        video_id = compute_video_id(local_video)
         was_reingest = db_instance.get_video(video_id) is not None

         # Validate (with-context variant surfaces ffprobe_meta for the report)
         print("[CLI] Validating...")
-        val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
+        val_results, val_context = registry.run_all_with_context(local_video, global_config)
         failures = [r for r in val_results if not r.passed]
```

```
         # Save video status
@@ -619,7 +634,7 @@ def main():
             segmenter_actual = seg_name
             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
             segmenter = segmenter_cls(db_instance, global_config)
-            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
+            result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
             segmentation_ran = True

             if isinstance(result, Failure):
```

14. assistant (2026-06-21T07:57:22.794Z)

Now let me look at the worker's already-shipped pattern and the path-safety helpers.

15. user (2026-06-21T07:57:24.235Z)

```
80      def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
81          with open(path, "w") as f:
82              json.dump({
83                  "video_id": video_id,
84                  "status": status,
85                  "segment_count": len(segments),
86                  "segments": [{"start": float(s.get("start_time", 0.0)),
87                                "end": float(s.get("end_time", 0.0))} for s in segments],
88              }, f, indent=2)
89
90
91      def cmd_infer(args: argparse.Namespace) -> int:
92          config = load_config(args.config) if args.config else load_config()
93          _apply_overrides(config, args.set)
94          storage_cfg = config.storage.model_dump()
95
96          scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
97          os.makedirs(scratch, exist_ok=True)
98          config.output.output_dir = scratch
99
100         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
101         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
102         print(f"[worker] pulled {args.video_uri} -> {local_video}")
103
104         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
105         video_id = compute_video_id(local_video)
106
107         # 3. VALIDATE (same registry as the main CLI).
108         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
109         failures = [r for r in val_results if not r.passed]
110         db = Database(os.path.join(scratch, config.output.db_name))
111         db.add_video(video_id, local_video, "failed" if failures else "processed",
112                      [r.model_dump() for r in val_results])
113
114         segments: List[dict] = []
115         segmenter_actual = None
116         segmentation_ran = False
117         status = "failed"
118         try:
119             if failures:
```

16. user (2026-06-21T07:57:24.564Z)

```
1       """Path-safety helpers for the API boundary (untrusted request fields)."""
2       import os
3       from typing import Optional
4
5
6       def safe_output_filename(name: str) -> str:
7           """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9           `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12          """
13          if not name or not name.strip():
14              raise ValueError("output_filename must not be empty")
15          if "\x00" in name:
16              raise ValueError("output_filename must not contain a null byte")
17          if "/" in name or "\\" in name:
18              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21          return name
22
23
24      def resolve_under_root(path: str, root: str) -> str:
25          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26          rp = os.path.realpath(path)
27          rr = os.path.realpath(root)
28          try:
29              contained = os.path.commonpath([rp, rr]) == rr
30          except ValueError:  # different drives on Windows
31              contained = False
32          if not contained:
33              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34          return rp
35
36
37      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
41          so the local 'point it at any of my videos' workflow is preserved. Setting an
42          allowed_root (a future hosted/tenant deployment) enforces containment.
43          """
44          if "\x00" in path:
45              raise ValueError("video_path must not contain a null byte")
46          if allowed_root:
47              resolve_under_root(path, allowed_root)
48
```

17. assistant (2026-06-21T07:57:26.926Z)

Now let me read the new test file and the api endpoint tests to understand the intended behavior.

18. user (2026-06-21T07:57:27.371Z)

```
1       """Input acquisition for COMPUTE_DECOUPLED_SERVING M2 ? resolve_input_uri /
input_is_local /
```

```
2       acquire_local_input.
3
4       The hard guarantee: the default config (input_backend='local', input_root='') is a byte-
for-byte
5       identity passthrough for every input path used today (no fetch, no copy). A bucket/Drive
source is
6       routed through storage.fetch to a local scratch file. All offline (the remote backend is
a
7       monkeypatched fetch ? no boto3/google-cloud-storage, no network).
8
9       OS note: os.path.isabs is platform-dependent, so the absolute-path cases use a POSIX-
style
10      '/data/...' (absolute on BOTH Windows and POSIX) and Windows backslash paths only with
input_root=''
11      (branch 5, OS-independent) ? the full suite runs cross-OS in CI.
12      """
13      import os
14
15      import pytest
16
17      from backend import storage
18      from backend.storage import acquire_local_input, input_is_local
19      from backend.storage.ref import resolve_input_uri
20
21
22      # ---------------------------------------------------------------- resolve_input_uri
23      @pytest.mark.parametrize("raw,ib,ir,expected", [
24          # 1. explicit scheme always wins (even over a non-local backend + root)
25          ("gs://b/x.mp4", "local", "", "gs://b/x.mp4"),
26          ("s3://b/x.mp4", "gcs", "gs://r", "s3://b/x.mp4"),
27          ("local://abs/x.mp4", "gcs", "gs://r", "local://abs/x.mp4"),
28          ("gdrive://Sharing/x.mp4", "local", "", "gdrive://Sharing/x.mp4"),
29          # 2. an anchored local path (leading slash, or drive prefix) is always local, never
joined ?
30          #    OS-independently (not via os.path.isabs, which differs on Windows+py3.13)
31          ("/data/x.mp4", "gcs", "gs://r", "/data/x.mp4"),
32          ("\\\\server\\share\\x.mp4", "gcs", "gs://r", "\\\\server\\share\\x.mp4"),
33          ("C:\\v\\x.mp4", "gcs", "gs://r", "C:\\v\\x.mp4"),
34          ("C:/v/x.mp4", "gcs", "gs://r", "C:/v/x.mp4"),
35          # 3. a bare/relative name joins under a non-empty input_root
36          ("x.mp4", "local", "gs://bucket/videos", "gs://bucket/videos/x.mp4"),
37          ("sub/x.mp4", "local", "gs://bucket/videos/", "gs://bucket/videos/sub/x.mp4"),  #
root trailing slash
38          # 4. input_backend prefixes a bare bucket/key when no input_root
39          ("bucket/x.mp4", "gcs", "", "gcs://bucket/x.mp4"),
40          ("bucket/x.mp4", "s3", "", "s3://bucket/x.mp4"),
41          # 5. bare/relative local path ? today's behaviour, unchanged
42          ("x.mp4", "local", "", "x.mp4"),
43          ("rel/sub/x.mp4", "local", "", "rel/sub/x.mp4"),
44          ("C:\\v\\x.mp4", "local", "", "C:\\v\\x.mp4"),  # Windows path, default config:
unchanged (OS-independent)
45      ])
46      def test_resolve_input_uri(raw, ib, ir, expected):
47          assert resolve_input_uri(raw, input_backend=ib, input_root=ir) == expected
48
49
50      def test_resolve_input_uri_default_is_identity():
51          for p in ("/data/clip.mp4", "rel/clip.mp4", "clip.mp4", "gs://b/k", "local://x"):
52              assert resolve_input_uri(p) == p
53
54
55      # ---------------------------------------------------------------- input_is_local
56      def test_input_is_local_true_for_local_paths():
57          assert input_is_local("/data/x.mp4") is True
58          assert input_is_local("rel/x.mp4") is True
```

```
59          assert input_is_local("local://x.mp4") is True
60
61
62      def test_input_is_local_false_for_buckets():
63          assert input_is_local("gs://b/x.mp4") is False
64          assert input_is_local("s3://b/x.mp4") is False
65          assert input_is_local("gdrive://Sharing/x.mp4") is False
66
67
68      def test_input_is_local_follows_input_root():
69          cfg = {"input_backend": "local", "input_root": "gs://bucket/videos"}
70          assert input_is_local("x.mp4", cfg) is False           # bare name resolves into the
bucket
71          assert input_is_local("/abs/x.mp4", cfg) is True        # absolute path stays local
72          assert input_is_local("gs://other/x.mp4", cfg) is False
73
74
75      def test_input_is_local_accepts_pydantic_storage_config():
76          from backend.config.models import StorageConfig
77          sc = StorageConfig(input_root="gs://bucket/videos")
78          assert input_is_local("x.mp4", sc) is False
79          assert input_is_local("/abs/x.mp4", sc) is True
80
81
82      # ---------------------------------------------------------------- acquire_local_input
83      def test_acquire_local_input_local_identity(tmp_path):
84          p = tmp_path / "v.mp4"
85          p.write_bytes(b"abc")
86          # Identity passthrough ? returns the SAME string, no copy, no temp dir.
87          assert acquire_local_input(str(p)) == str(p)
88
89
90      def test_acquire_local_input_relative_local_identity():
91          assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
92
93
94      def _capturing_fetch(captured):
95          def _f(uri, dst=None, *, config=None, overwrite=False):
96              captured["uri"] = uri
97              captured["dst"] = dst
98              captured["config"] = config
99              if dst:
100                 with open(dst, "wb") as fh:
101                     fh.write(b"remote-bytes")
102             return dst or uri
103         return _f
104
105
106     def test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
107         captured = {}
108         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
109         out = acquire_local_input("gs://bucket/videos/x.mp4", {"backend": "local", "gcs":
{}})
110         assert captured["uri"] == "gs://bucket/videos/x.mp4"          # exact URI routed to
fetch
111         assert os.path.basename(out) == "x.mp4"                       # named from the ref
112         assert os.path.isfile(out) and open(out, "rb").read() == b"remote-bytes"
113         # The whole storage settings dict is forwarded so per-backend (gcs/s3) config
reaches fetch.
114         assert captured["config"] == {"backend": "local", "gcs": {}}
115
116
117     def test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
118         captured = {}
119         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
```

```
120         acquire_local_input("x.mp4", {"input_root": "gs://bucket/videos"})
121         assert captured["uri"] == "gs://bucket/videos/x.mp4"          # resolved BEFORE
fetch
122
123
124     def test_acquire_local_input_never_fetches_for_local(monkeypatch):
125         def _boom(*a, **k):
126             raise AssertionError("fetch must not be called for a local input")
127         monkeypatch.setattr(storage, "fetch", _boom)
128         assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
129         assert acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
== "clip.mp4"
130
```

19. assistant (2026-06-21T07:57:30.111Z)

Now let me check the api endpoint tests diff, and look at the worker's `fetch` more carefully to compare the two paths (worker uses `storage.fetch` directly, M2 uses `acquire_local_input`).

20. user (2026-06-21T07:57:31.397Z)

```
diff --git a/tests/test_api_endpoints.py b/tests/test_api_endpoints.py
index 001c9fc..ebedddf 100644
--- a/tests/test_api_endpoints.py
+++ b/tests/test_api_endpoints.py
@@ -334,3 +334,61 @@ def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
     jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
     assert jobf["result"]["status"] == "failed"
     # empty-run similar (would preserve prior via reingest safety in both paths)
+
+
+# ---------------------------------------------------------------- M2: URI-aware input
+class _CaptureSeg:
+    """Records the video_path the segmenter actually received (M2: must be a LOCAL path)."""
+    last_path = None
+
+    def __init__(self, db, cfg):
+        self.db = db
+
+    def process_video(self, video_id, video_path, *a, **k):
+        _CaptureSeg.last_path = video_path
+        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
+        return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
{}}]
+
+
+def test_process_local_input_passes_identity_path(env):
+    """Default (local) input must reach the segmenter as the EXACT same path ? identity, no
copy
+    (the M2 zero-behaviour-change guarantee for today's local workflow)."""
+    client, db, tmp_path, mp = env
+    vid = _video(tmp_path)
+    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+    _CaptureSeg.last_path = None
+    r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})
+    assert r.status_code == 202
+    assert _CaptureSeg.last_path == str(vid)
+
+
+def test_process_local_missing_file_still_404(env):
+    """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is preserved
+    for local paths (only non-local URIs skip it)."""
+    client = env[0]
+    r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
```

```
+    assert r.status_code == 404
+
+
+def test_process_nonlocal_uri_skips_exists_and_fetches(env):
+    """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the worker
+    fetches it to a local scratch file and the segmenter sees that LOCAL path, not the URI."""
+    client, db, tmp_path, mp = env
+
+    def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+        with open(dst, "wb") as fh:
+            fh.write(b"fetched-video-bytes")
+        return dst
+
+    mp.setattr(m.storage, "fetch", fake_fetch)
+    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+    _CaptureSeg.last_path = None
+    r = client.post("/api/process", json={"video_path": "gs://bucket/videos/x.mp4",
"segmenter": "x"})
+    assert r.status_code == 202                              # NOT 404 ? non-local skips the
exists gate
+    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
+    assert job["status"] == "done"
+    assert _CaptureSeg.last_path is not None
+    assert not _CaptureSeg.last_path.startswith("gs://")     # a fetched LOCAL path, not the
URI
+    assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"
```

21. assistant (2026-06-21T07:57:36.996Z)

Now let me examine a key correctness concern: the worker uses `storage.fetch(uri, ..., config=storage_cfg)` where `storage_cfg = config.storage.model_dump()`. The M2 `acquire_local_input` forwards `sc` (the model_dump of storage config) as `config` to `fetch`. But `fetch` ? `get_storage(ref.backend, config=config)` ? `settings = dict(cfg.get(name, {}))`. So `config` must have a per-backend key like `cfg['gcs']`. Let me verify the StorageConfig model_dump shape includes those per-backend dicts, and check `get_storage`.

The `StorageConfig.model_dump()` includes `gcs`, `s3`, `gdrive`, `local` keys (lines 326-329). So `cfg.get('gcs', {})` works. Good ? consistent with worker.

Now let me check the edge cases the lens calls out. Let me verify the round-trip and the `input_root` join producing a valid parse_uri. Let me test a few cases directly.

22. user (2026-06-21T07:57:45.129Z)

```
raw=''                            ib=local  ir=''                -> uri=''
backend=local   bucket=''         key=''                  name=''
raw=''                            ib=gcs    ir='gs://bucket/v'    -> uri='gs://bucket/v/'
backend=gcs     bucket='bucket'   key='v/'                name='v'
raw='noext'                       ib=local  ir='gs://bucket/v'    -> uri='gs://bucket/v/noext'
backend=gcs     bucket='bucket'   key='v/noext'           name='noext'
raw='dir/'                        ib=local  ir='gs://bucket/v'    -> uri='gs://bucket/v/dir/'
backend=gcs     bucket='bucket'   key='v/dir/'            name='dir'
raw='x.mp4'                       ib=local  ir='gs://bucket/v'    -> uri='gs://bucket/v/x.mp4'
backend=gcs     bucket='bucket'   key='v/x.mp4'           name='x.mp4'
raw='x.mp4'                       ib=local  ir='gs://bucket/v/'   -> uri='gs://bucket/v/x.mp4'
backend=gcs     bucket='bucket'   key='v/x.mp4'           name='x.mp4'
raw='gs://b/x.mp4'                ib=gcs    ir=''                 -> uri='gs://b/x.mp4'
backend=gcs     bucket='b'        key='x.mp4'             name='x.mp4'
raw='gdrive://Shared/x.mp4'       ib=local  ir=''                 ->
uri='gdrive://Shared/x.mp4'             backend=gdrive  bucket=''         key='Shared/x.mp4'
name='x.mp4'
raw='gs://bucket'                 ib=local  ir=''                 -> uri='gs://bucket'
backend=gcs     bucket='bucket'   key=''                  name=''
raw='bucket'                      ib=gcs    ir=''                 -> uri='gcs://bucket'
```

```
backend=gcs     bucket='bucket'   key=''                    name=''
```

23. assistant (2026-06-21T07:57:53.155Z)

Interesting findings. Let me focus on the most important one: the **empty string with a non-local input_root** case. `resolve_input_uri('', 'gcs', 'gs://bucket/v')` ? ``'gs://bucket/v/'`` (key ends in `/`, name=``'v'``). And `gs://bucket` with no key ? name=``''``, so `acquire_local_input` falls back to ``'in.mp4'``. Let me trace the `acquire_local_input` for the empty-string + local-default case, which is the critical SAFE/identity guarantee.

For `acquire_local_input('', None)`: `resolve_input_uri('', 'local', '')` ? `''` (branch 5), `parse_uri('')` ? backend `local`, key `''`, returns `ref.key` = `''`. So an empty input returns empty ? identity, harmless (caller already validated existence). Good.

Now let me examine the more subtle divergence: the worker fetches with `dst=os.path.join(scratch, "in.mp4")` (fixed name), while `acquire_local_input` uses `ref.name or "in.mp4"`. Let me check whether the StubFetch and real backends honor the `dst` name or derive their own. Let me look at the local/gcs/s3 `fetch_to_local`.

24. user (2026-06-21T07:57:55.863Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 3612bd8..a31c9da 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -302,6 +302,16 @@ class StorageConfig(BaseModel):
     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
     # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
     output_root: str = ""
+    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
+    # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is read IN
PLACE
+    # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
+    # input name is resolved against ``input_root`` (e.g. ``input_root="gs://bucket/videos"`` +
"x.mp4"
+    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
+    # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the input
+    # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are
fetched to a
+    # scratch dir before processing (see ``backend.storage.acquire_local_input``).
+    input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+    input_root: str = ""
     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
=== ref.py and __init__.py are fully new additions (already read) ===
=== check: is resolve_input_uri / acquire used anywhere else? ===
backend/config/models.py:312:    # scratch dir before processing (see
``backend.storage.acquire_local_input``).
backend/main.py:159:    local_video = storage.acquire_local_input(req.video_path,
getattr(global_config, "storage", None))
backend/main.py:278:    if storage.input_is_local(req.video_path, getattr(global_config,
"storage", None)):
backend/main.py:594:        if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
backend/main.py:597:        local_video = storage.acquire_local_input(args.video_path,
storage_cfg)
backend/storage/__init__.py:20:from backend.storage.ref import StorageRef, StorageStat,
parse_uri, resolve_input_uri
backend/storage/__init__.py:23:    "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
"resolve_input_uri",
```

```
backend/storage/__init__.py:25:    "acquire_local_input", "input_is_local",
backend/storage/__init__.py:38:def input_is_local(raw: str, storage_config: Any = None) -> bool:
backend/storage/__init__.py:44:    uri = resolve_input_uri(raw,
input_backend=sc.get("input_backend", "local"),
backend/storage/__init__.py:49:def acquire_local_input(raw: str, storage_config: Any = None, *,
backend/storage/__init__.py:59:    uri = resolve_input_uri(raw,
input_backend=sc.get("input_backend", "local"),
backend/storage/ref.py:71:def resolve_input_uri(raw: str, *, input_backend: str = "local",
input_root: str = "") -> str:
```

25. assistant (2026-06-21T07:58:04.583Z)

Now a critical concern about consistency between `input_is_local` and `acquire_local_input`. Both call `resolve_input_uri` with the same args, so they resolve identically. Good ? they're consistent by construction.

Now the key divergence concern: in `_execute_processing` (API path), the **existence gate** is in `process_video` (the request handler, line 278), but `acquire_local_input` runs in `_execute_processing` (line 159) which is called by the **job worker** later. Let me verify these are wired such that `acquire_local_input` is actually called and check the `getattr(global_config, "storage", None)` shape passed. `global_config.storage` is a `StorageConfig` pydantic model ? `_storage_dict` calls `model_dump()`. Good.

Now let me check a real subtlety: the `process_video` request handler validates with `validate_input_path` using `os` import. Let me confirm `os` is still imported in main.py and check the full `process_video` handler context around the existence gate.

26. user (2026-06-21T07:58:04.752Z)

```
258
259
260     @app.post("/api/process", status_code=202)
261     def process_video(req: ProcessRequest):
262         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
263
264         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
265         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
266         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
267         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
268         """
269         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
270         try:
271             validate_input_path(req.video_path, allowed_root=allowed_root)
272         except ValueError as e:
273             raise HTTPException(status_code=400, detail=str(e)) from e
274         # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive
URI can't be
275         # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it
(a fetch miss
276         # fails the job loudly). validate_input_path above still runs for every input: its
null-byte guard
277         # always applies, and a configured allowed_video_root (hosted mode) rejects a non-
local URI.
278         if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
279             if not os.path.exists(req.video_path):
280                 raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
281         if req.segmenter and not segmenter_registry.get(req.segmenter):
282             available = list(segmenter_registry.list_available().keys())
283             raise HTTPException(
284                 status_code=400,
285                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
286             )
```

```
287
288         job_id = uuid.uuid4().hex
289         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
290                                       req.player_pool, req.max_frames):
291             raise HTTPException(status_code=500, detail="Failed to persist job record.")
292         _get_executor().submit(_drain_due_jobs)
293         return JSONResponse(
294             status_code=202,
295             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
296         )
297
```

27. assistant (2026-06-21T07:58:08.100Z)

Now there's an interaction with `allowed_video_root` (hosted mode) that's worth examining closely. The comment claims "a configured `allowed_video_root` (hosted mode) rejects a non-local URI". Let me verify that claim by testing `validate_input_path` with a `gs://` URI and an `allowed_root`. This is a security-relevant claim.

28. user (2026-06-21T07:58:11.181Z)

```
'gs://bucket/secret/x.mp4'          -> rejected: path 'gs://bucket/secret/x.mp4' escapes the
allowed root 'C:\\Users\\avidu\\AppData\\Local\\Temp\\tmpzc74f8jz'
's3://b/x.mp4'                      -> rejected: path 's3://b/x.mp4' escapes the allowed root
'C:\\Users\\avidu\\AppData\\Local\\Temp\\tmpzc74f8jz'
'gdrive://Shared/x.mp4'             -> rejected: path 'gdrive://Shared/x.mp4' escapes the
allowed root 'C:\\Users\\avidu\\AppData\\Local\\Temp\\tmpzc74f8jz'
```

29. assistant (2026-06-21T07:58:20.410Z)

Good ? that claim holds: in hosted mode, a bucket URI is rejected by `validate_input_path` (the `realpath` of `gs://...` doesn't live under the allowed root). So the existence-gate skip for non-local doesn't open a hosted-mode hole. (On Windows realpath treats it as a relative path under cwd; on POSIX similar ? either way it escapes the root.)

Now let me examine the most important divergence the lens asks about: **worker vs main producing different bytes**. The worker (`cmd_infer`) calls `storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg)` ? it always passes `args.video_uri` raw to fetch, WITHOUT going through `resolve_input_uri`. The main path goes through `acquire_local_input` ? `resolve_input_uri`. So for a **bare relative name with a non-local input_backend/input_root configured**, the two paths diverge:

- Worker: `fetch("x.mp4", ...)` ? `parse_uri("x.mp4")` ? local backend ? identity (returns `"x.mp4"` or `dst`?). Let me check what `LocalStorage.fetch_to_local` does with a relative path + dst.
- Main: `acquire_local_input("x.mp4", input_root="gs://bucket/v")` ? fetches `gs://bucket/v/x.mp4`.

But the worker is a separate, pre-existing entry point (the async job model). For M2, the API job path goes through `_execute_processing`, not `cmd_infer`. Let me verify which worker actually runs the API jobs ? is it `cmd_infer` or `_execute_processing`?

30. user (2026-06-21T07:58:21.806Z)

```
backend\main.py:68:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`,
`_get_executor`,
backend\main.py:76:    _drain_due_jobs,
backend\main.py:140:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
backend\main.py:267:    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-
scheduling loop).
backend\main.py:292:    _get_executor().submit(_drain_due_jobs)
backend\main.py:669:        _get_executor().submit(_drain_due_jobs)
backend\infrastructure\database.py:211:        # re-claims it via `claim_due_job` when due ?
```

```
with NO master node: the table IS
backend\infrastructure\database.py:561:        worker. `claim_due_job` re-picks it when due ?
cooperative, no master. ``next_poll_at`` is
backend\infrastructure\database.py:588:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
backend\api\job_worker.py:4:The worker DRAINS the jobs table via `claim_due_job` and honours
WAIT_AND_RESUME: a job
backend\api\job_worker.py:9:every cross-function seam ? `db_instance`, `global_config`,
`_execute_processing`,
backend\api\job_worker.py:45:    releases the thread, and re-claims the job when due
(`claim_due_job`). No work to date
backend\api\job_worker.py:46:    is the segmenter's `_execute_processing`, run UNDER the
destroy-after-confirm contract.
backend\api\job_worker.py:61:    re-claimed by `claim_due_job` (queued OR resumed-from-waiting)
runs identically to
backend\api\job_worker.py:73:    """Run ONE job already claimed (`status='running'`) by
`claim_due_job`, recording its
backend\api\job_worker.py:86:        result = _main._execute_processing(
backend\api\job_worker.py:100:def _drain_due_jobs() -> int:
backend\api\job_worker.py:112:        job = _main.db_instance.claim_due_job()
backend\api\job_worker.py:145:    timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
```

31. assistant (2026-06-21T07:58:29.602Z)

Confirmed: the API job worker (`backend/api/job_worker.py:86`) runs `_main._execute_processing`, which is the M2-instrumented path. The `backend.worker.__main__:cmd_infer` is the **separate Cloud Run/CLI worker** (a distinct entry point for the decoupled-compute track), not the API job drainer. So there are genuinely two fetch entry points:

1. `backend.main` (API + CLI) ? `acquire_local_input` ? `resolve_input_uri` (M2 new).
2. `backend.worker.__main__:cmd_infer` ? `storage.fetch(uri)` directly (NO `resolve_input_uri`).

This is a real divergence the lens flags. The worker takes a full URI (`video_uri` positional arg) and never applies `input_root`/`input_backend`. That's arguably fine (the worker is given a fully-qualified URI by the dispatcher), but it means `input_root`/`input_backend` config is **silently ignored by the worker path** ? a bare name + `input_root=gs://...` resolves to the bucket via `main` but stays local via the worker. Let me verify the worker doesn't call `resolve_input_uri`.

32. user (2026-06-21T07:58:44.352Z)

```
.....................................                                    [100%]
42 passed in 11.22s
```

33. assistant (2026-06-21T07:58:58.881Z)

Tests pass. Now let me verify one important safety concern: the `compute_video_id` now hashes `local_video` instead of `req.video_path`. For a local input these are identical (identity passthrough), so `video_id` is byte-identical to before ? that's the SAFE guarantee. Confirmed for local: `acquire_local_input(path) == path` (returns `ref.key` which is the same string).

But wait ? there's a subtle case. For a `local://` URI input (explicit scheme), `parse_uri("local://C:/v/x.mp4")` ? key = `"C:/v/x.mp4"`, and `acquire_local_input` returns `ref.key` = `"C:/v/x.mp4"` (stripped of the `local://` prefix). So if someone passed `local://...`, the resolved path differs from the raw (prefix stripped) ? but that's correct/desired (it's still local, identity to the underlying file). And `req.video_path` (raw URI) is preserved for the DB record. That's fine.

Now let me check the critical default-config identity for the **CLI path** carefully. There's an ordering subtlety: at main.py:594, `input_is_local(args.video_path, storage_cfg)` is checked first; if local AND not exists ? fail. Then `acquire_local_input`. For the default local config, `input_is_local` returns True, `os.path.exists` runs (preserving today's check), then `acquire_local_input` returns the same path. Byte-identical. Good.

Let me now verify the `gs://bucket` (no key) ? `name=''` ? fallback `"in.mp4"` behavior in `acquire_local_input`, and check whether `fetch` for gcs with an empty key would be a problem. Also let me double-check the trailing-slash key case (`v/` ? name `v`). Let me look at whether the worker's `model_dump()` storage shape and `acquire_local_input`'s forwarded dict diverge in what `fetch`?`get_storage` reads for the backend selection.

In `acquire_local_input`, `fetch(uri, ..., config=sc)` where `sc = storage_config.model_dump()`. Inside `fetch`, `get_storage(ref.backend, config=sc)`. Note: `get_storage` uses `ref.backend` (from the URI), NOT `sc['backend']`. So even though `sc['backend']` might be `'local'` (the output backend) while the input is `gs://`, `fetch` correctly uses the URI-derived backend `gcs`, and reads `sc['gcs']` settings. This matches the worker exactly (worker also passes `config.storage.model_dump()` and `fetch` keys off the URI). Consistent. Good.

Let me do one final check on the `input_root` join when `input_root` itself has an explicit scheme that's `local://` and whether the join still round-trips, plus the `gdrive://` path with input_root.

34. user (2026-06-21T07:59:03.549Z)

```
'x.mp4' ib=local ir='local://data/videos'     -> 'local://data/videos/x.mp4'
backend=local key='data/videos/x.mp4' name='x.mp4'
'x.mp4' ib=local ir='gdrive://My Drive/clips' -> 'gdrive://My Drive/clips/x.mp4'
backend=gdrive key='My Drive/clips/x.mp4' name='x.mp4'
'x.mp4' ib=gdrive ir=''                        -> 'gdrive://x.mp4'
backend=gdrive key='x.mp4' name='x.mp4'
--- acquire_local_input identity for local:// root (should NOT fetch, returns local key) ---
'data/videos/x.mp4'
```

35. assistant (2026-06-21T07:59:10.593Z)

All the join cases round-trip correctly through `parse_uri`. The `local://` root case correctly resolves to a local backend and returns the bare key (no fetch).

Now one last thing ? let me confirm the empty-string-with-non-local-root edge (`'' + gs://bucket/v` ? `gs://bucket/v/`, name `v`). In `acquire_local_input` this would `fetch('gs://bucket/v/', dst=.../v)` ? a directory-like URI. That's a degenerate input (empty filename), not a real scenario, and the existence gate at the API layer won't catch it because `input_is_local` returns False. But an empty `video_path` would already be caught upstream ? let me check if `ProcessRequest` or `validate_input_path` rejects empty. `validate_input_path('')` only checks null bytes ? empty passes. But the worker would fail loudly on fetch. This is an extreme edge (empty filename + remote root), low severity.

Let me verify the full suite collects cleanly for the touched modules and run a broader sanity check (config + storage tests).

36. user (2026-06-21T07:59:12.978Z)

```
ERROR: file or directory not found: tests/test_storage_layer.py


no tests ran in 0.00s
```

37. user (2026-06-21T07:59:32.288Z)

```
test_storage.py
---
.....................................................                    [100%]
55 passed, 1041 deselected in 8.12s
```

38. assistant (2026-06-21T07:59:41.267Z)

All green. Let me do a final review of the consistency between `input_is_local` and `acquire_local_input` for the one case that could bite: a `local://` URI. `input_is_local("local://x.mp4")` ? resolve ? `"local://x.mp4"` (explicit scheme, branch 1) ?

parse ? backend `local` ? True. And `acquire_local_input("local://x.mp4")` ? returns `ref.key` = `"x.mp4"`. So existence-gate runs against the raw `"local://x.mp4"` string in the API handler (`os.path.exists("local://x.mp4")`), which would be **False** on disk even if `x.mp4` exists ? a 404 false-negative.

Wait, let me check this carefully. This is a real inconsistency: the existence check at main.py:279 uses `req.video_path` (the raw `local://...` string), not the resolved key.

39. user (2026-06-21T07:59:44.834Z)

```
raw = local://C:/Users/avidu/AppData/Local/Temp/tmpdlb828eq/x.mp4
input_is_local(raw) = True
os.path.exists(raw) = False
acquire returns = C:/Users/avidu/AppData/Local/Temp/tmpdlb828eq/x.mp4
os.path.exists(acquired) = True
```

40. assistant (2026-06-21T08:00:00.210Z)

Confirmed a real bug. A `local://<abs path>` input that genuinely exists on disk is rejected with a spurious 404 (and in the CLI path, "Video file not found") because the existence gate stats the raw `local://...` string instead of the resolved local key. The same happens for the CLI at main.py:594.

This is the kind of bug worth flagging, though `local://` inputs aren't today's default path (bare paths are). Severity is minor-to-major depending on whether `local://` is a documented input form. The module docstring and `parse_uri` explicitly document `local://<path>` as a supported local form (ref.py:6), and the M2 tests even assert `input_is_local("local://x.mp4") is True`. So this is a documented input form that's mishandled by the gate.

Let me check whether this is a regression or pre-existing. Before M2, the gate was unconditional `os.path.exists(req.video_path)` ? so `local://...` was *also* rejected before (it was never resolved). So this isn't a *new* regression; M2 preserves the (arguably already-broken) behavior for `local://`. But M2 newly *added* `input_is_local` which returns True for `local://`, creating the expectation that local inputs work ? yet the gate still checks the unresolved string. The clean fix is to stat the resolved path. I'll flag it as minor (documented form, but not the default/common path, and pre-existing for the raw-`os.path.exists` part).

Let me also double check: does the default config path (bare path, the actual today behavior) have ANY divergence? No ? for a bare path, raw == resolved key, existence gate and acquire both use the same string. The SAFE/default-OFF guarantee holds for the common case.

Now let me confirm there are no lint/type issues introduced and check the `Any` typing import usage is fine.

41. user (2026-06-21T08:00:01.334Z)

```
All checks passed!
=== mypy ===
Success: no issues found in 2 source files
```

42. assistant (2026-06-21T08:00:11.576Z)

Lint and types clean. Let me make one final consistency verification: that `input_is_local` and `acquire_local_input` agree on the backend for the `gs://bucket` (no key) edge and that the `name or "in.mp4"` fallback truly only triggers when key has no basename. I've already confirmed `gs://bucket` ? name `''` ? fallback `in.mp4`. That's a sensible fallback for a keyless bucket URI (degenerate, but won't crash).

I have enough to finalize. Let me summarize the M2 review.

Summary of findings:

The M2 change is well-constructed and the core SAFE/default-OFF guarantee holds: for the default config (`input_backend='local'`, `input_root=''`) and bare/absolute local paths ? today's actual

input forms ? `resolve_input_uri` is an identity passthrough, `acquire_local_input` returns the same string with no copy, and `compute_video_id`/validators/segmenter see the byte-identical path. `input_is_local` and `acquire_local_input` are consistent by construction (both delegate to the same `resolve_input_uri` call). The storage-config shape forwarded to `fetch` matches the worker's `config.storage.model_dump()` pattern, and `fetch` keys the backend off the URI (not `sc['backend']`), so input and output backends don't collide. The hosted-mode `allowed_video_root` gate still rejects bucket URIs, so skipping the local existence-check for non-local inputs opens no security hole. Edge cases (trailing-slash root, no-extension name, `gdrive://`, keyless bucket, empty string) all round-trip through `parse_uri` without crashing.

One real bug: a `local://<abs-path>` input (a documented local input form, asserted local by the M2 tests) is mis-stat'd by the existence gate, yielding a spurious 404/"not found" even when the file exists.

43. user (2026-06-21T08:00:30.977Z)

Structured output provided successfully

44. assistant (2026-06-21T08:00:43.833Z)

M2 review complete. Summary of what I verified and found:

The SAFE / default-OFF guarantee holds. For the default config (`input_backend='local'`, `input_root=''`) and bare/absolute local paths ? today's actual input forms ? `resolve_input_uri` is identity, `acquire_local_input` returns the same string with no copy, and `compute_video_id`/validators/segmenter all receive the byte-identical path. `input_is_local` and `acquire_local_input` are consistent by construction (both delegate to one `resolve_input_uri` call). The storage dict forwarded to `fetch` matches the worker's `config.storage.model_dump()` pattern, `fetch` keys the backend off the URI (not `sc['backend']`), and the hosted-mode `allowed_video_root` gate still rejects bucket URIs ? so skipping the local existence check for non-local inputs opens no security hole. Every enumerated edge case (empty string, no-extension name ? `in.mp4` fallback, key ending `/`, root with/without trailing slash, explicit scheme matching the backend, `gdrive://`, `gs://bucket` with no key) round-trips through `parse_uri` without crashing. Tests pass (42 in the two M2 files, 55 across storage+config+input), ruff and mypy are clean.

Two findings (returned via StructuredOutput):

1. **minor / high-confidence** ? `backend/main.py:278-280` (API) and `:594` (CLI): a `local://<abs-path>` input ? a documented local form that `input_is_local` reports as local ? is stat'd as the raw `local://...` string instead of the resolved key, so an existing file is wrongly 404'd / "not found". The bare-path default path is unaffected; only the explicit `local://` scheme is hit. Fix: resolve once via `acquire_local_input` and stat that, reusing the value for hashing/validation.

2. **nit / high-confidence** ? `backend/worker/__main__.py:101`: the decoupled-compute worker bypasses `resolve_input_uri`, so `input_root`/`input_backend` apply on the API/CLI path but are inert on the worker path. No bytes-divergence for fully-qualified URIs (the only thing the dispatcher hands the worker), and the API job drainer uses the instrumented `_execute_processing`, not `cmd_infer` ? so it doesn't block M2. Flagged for the design record.

Relevant files: `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/storage/ref.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/storage/__init__.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_input_source.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_api_endpoints.py`.